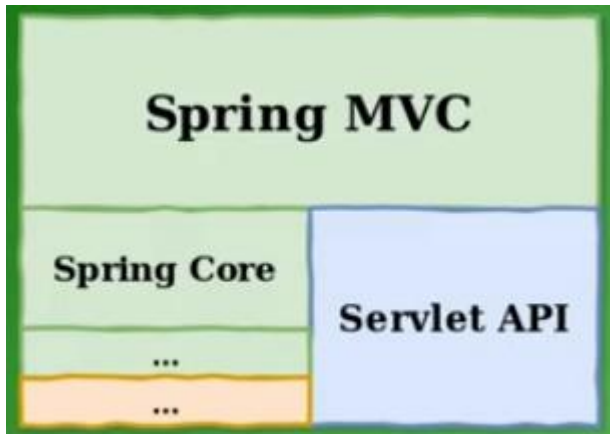


Spring

E' nato come tentativo di rendere lo sviluppo J2EE meno doloroso.



Introduzione

Con l'evoluzione di java ee si sono presentati tre possibili scenari a nostra disposizione:

- 1) Spring è utilizzato da solo ma dipende da jee (servlet API)
Ci sono casi in cui parti di Spring, dipendono da JEE, ad esempio Spring MVC con il quale può costruire un servizio Rest partendo da una servlet; questo comporta che un'applicazione Spring MVC per girare deve essere deployata in un servlet container (TOMCAT).
- 2) Mescolare Spring e JEE
Ci sono casi in cui è possibile mescolare parti di Spring e implementazione dello JEE, ad esempio potreste creare un'applicazione che utilizza Spring core e Jax-RS (specifica Jee per creare Servizi Rest) .
[Applicativi di esempio](#)
- 3) Nell'Utilizzo dei Bean Spring e Java EE sono in contrapposizione.
Spring (IoC Inversion of Control)
CDI (Context and Dependency Injection)
Sono tecnologie simili ma non possono essere mescolate.

Non è necessario conoscere JEE per usare Spring, il punto a favore di Spring è che ha una documentazione ufficiale, ed è più facile trovare informazioni su Spring.

SpringBoot

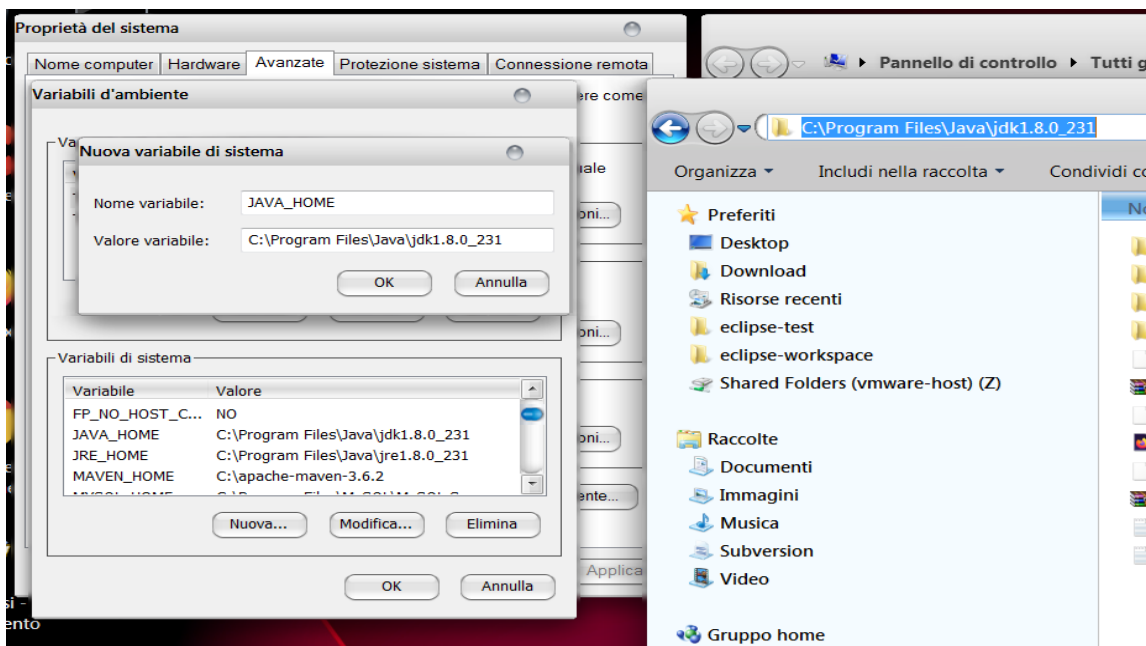
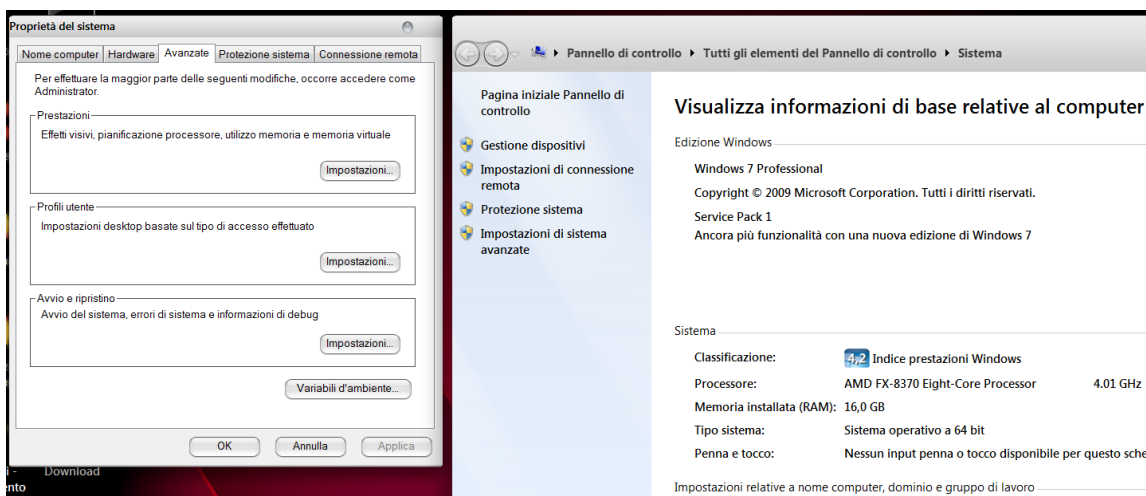
- Per costruire un'applicazione potete utilizzare un approccio Spring normale o SpringBoot. SpringBoot ha una serie di tools e utility che hanno il compito di facilitare lo sviluppo.

Spring MVC - Environment Setup

Questo capitolo è una guida su come configurare il Vostro computer in modo da essere pronti a realizzare un progetto Spring Framework. Andremo, quindi, a installare una **JDK**, **Tomcat** ed **Eclipse**

Step 1 - Setup Java Development Kit (JDK)

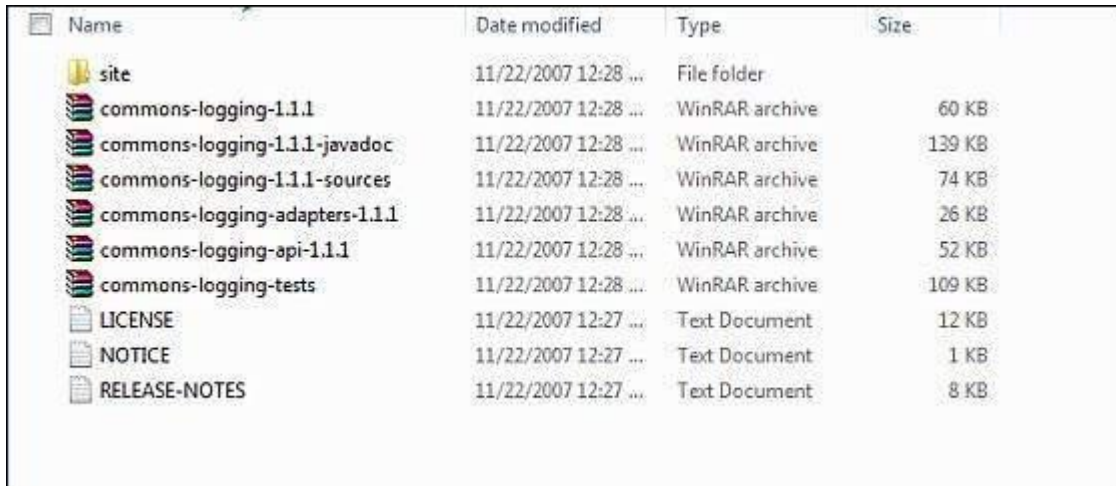
Scaricate sempre le ultime versioni di tutto, andate sul sito di Oracle Java – [Java SE Downloads](#). Da qui scaricate ed installate la JDK e seguite le istruzioni di configurazioni durante l'installazione. Una volta terminato andate nelle variabili d'ambiente di Windows e aggiungete una nuova variabile d'ambiente per la JDK ed una per la JRE.



Fate Nuova e aggiungete JAVA_HOME e JRE_HOME

Step 2 – Install Apache Common Logging API

Scarica dal sito Apache l'ultima versione di Apache Commons Logging API da qui (<https://commons.apache.org/logging/>.) Una volta scaricato il file compresso, decomprimi il contenuto in c:\

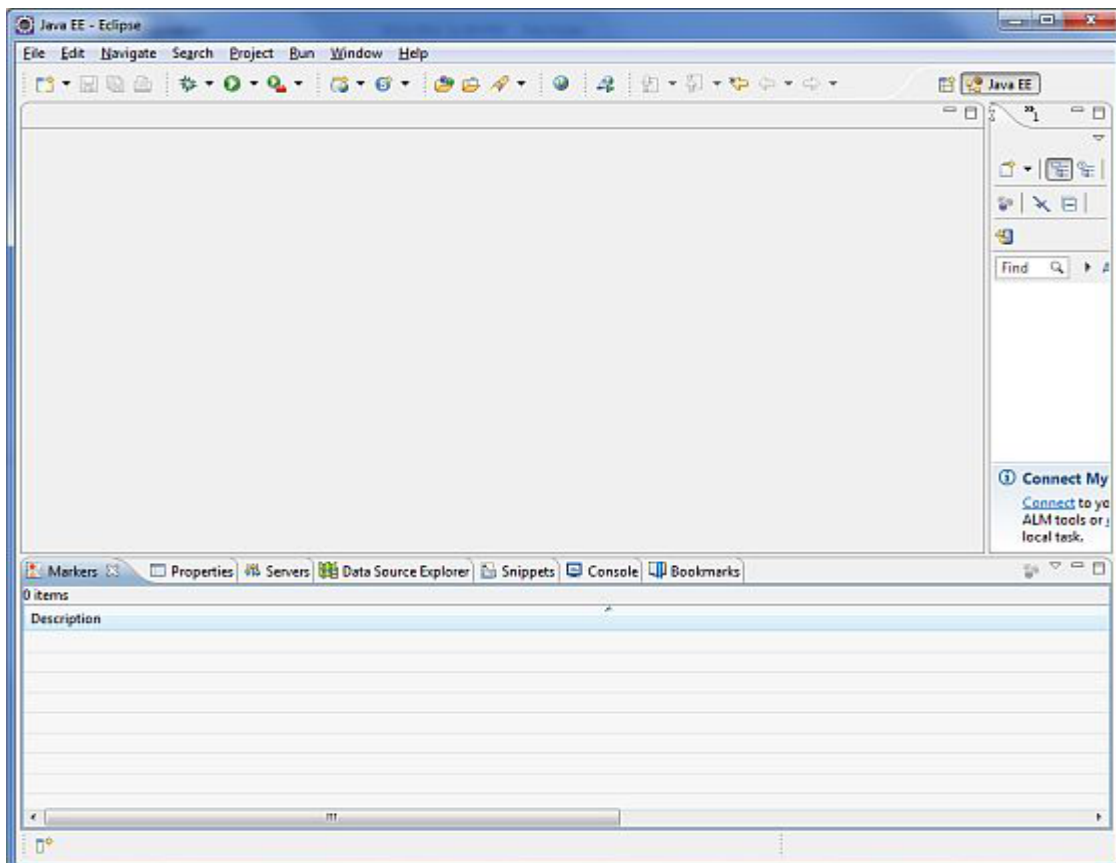


Name	Date modified	Type	Size
site	11/22/2007 12:28 ...	File folder	
commons-logging-1.1.1	11/22/2007 12:28 ...	WinRAR archive	60 KB
commons-logging-1.1.1-javadoc	11/22/2007 12:28 ...	WinRAR archive	139 KB
commons-logging-1.1.1-sources	11/22/2007 12:28 ...	WinRAR archive	74 KB
commons-logging-adapters-1.1.1	11/22/2007 12:28 ...	WinRAR archive	26 KB
commons-logging-api-1.1.1	11/22/2007 12:28 ...	WinRAR archive	52 KB
commons-logging-tests	11/22/2007 12:28 ...	WinRAR archive	109 KB
LICENSE	11/22/2007 12:27 ...	Text Document	12 KB
NOTICE	11/22/2007 12:27 ...	Text Document	1 KB
RELEASE-NOTES	11/22/2007 12:27 ...	Text Document	8 KB

Aggiungi come per la JDK la directory sopra nelle PATH variable

Step 3 - Setup Eclipse IDE

Adesso andiamo ad installare l'IDE che è un tool che ci aiuta nello sviluppo del nostro progetto. To install Trovate eclipse qui, <https://www.eclipse.org/downloads/>. Anche qui di solito si scarica un file compresso da estrarre in C:\



Step 4 - Setup Spring Framework Libraries

Anche Spring Framework va scaricato e poi installato

- Potete scaricarlo da qui <https://repo.spring.io/release/org/springframework/spring>.
- Una volta scaricato **spring-framework-4.3.1.RELEASE-dist.zip (ad esempio)** su Windows Machine troverete una struttura di cartelle windows in C:\spring some segue:

Name	Date modified	Type	Size
docs	4/22/2015 2:44 PM	File folder	
libs	4/22/2015 2:45 PM	File folder	
schema	4/22/2015 2:45 PM	File folder	
license	4/22/2015 2:42 PM	Text Document	15 KB
notice	4/22/2015 2:42 PM	Text Document	1 KB
readme	4/22/2015 2:42 PM	Text Document	1 KB

Tutte le librerie di Spring sono in **C:\spring\libs**.

Anche in questo caso dobbiamo aggiungere le librerie nel PATH Variable di Windows, in modo da informare Eclipse su dove si trova tutto, la jdk, spring e common Logging

Spring IoC

La gestione dei Beans

E' la parte del core di Spring che si occupa della gestione dei Bean. I bean sono delle particolari classi di java che vengono istanziate dal programmatore (new Class()) ma direttamente dal framework stesso.

Questo permette allo Spring framework di decidere in autonomia quante istanze di una classe creare

Beans

~~new MyClass()~~



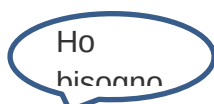
Un' istanza per tutt'applicazione



Un' istanza per ogni sessione utente, apertura di un nuova
finestra del browser



Un' istanza per ogni richiesta http, il
ricaricamento della pagina





Ogni bean dichiara di quali altri Bean ha bisogno, ed è il framework a passarglieli.

Tutti i bean passati Spring MVC sono dei singleton (Pattern che consente di avere una sola istanza di una classe).

SpringBoot

Per generare lo scheletro di un progetto Springboot bisogna andare a questo sito:

<https://start.spring.io>

Ovviamente esistono plugins per IDE che fanno la stessa cosa all'interno del vostro ambiente di sviluppo

[Creazione di un progetto SpringBoot con Eclipse e Maven](#)

Primo progetto in Spring

Quello che viene generato nel sito start.spring.io è un progetto maven e nel pom vengono automaticamente generate le dipendenze e il plugin per Springframework

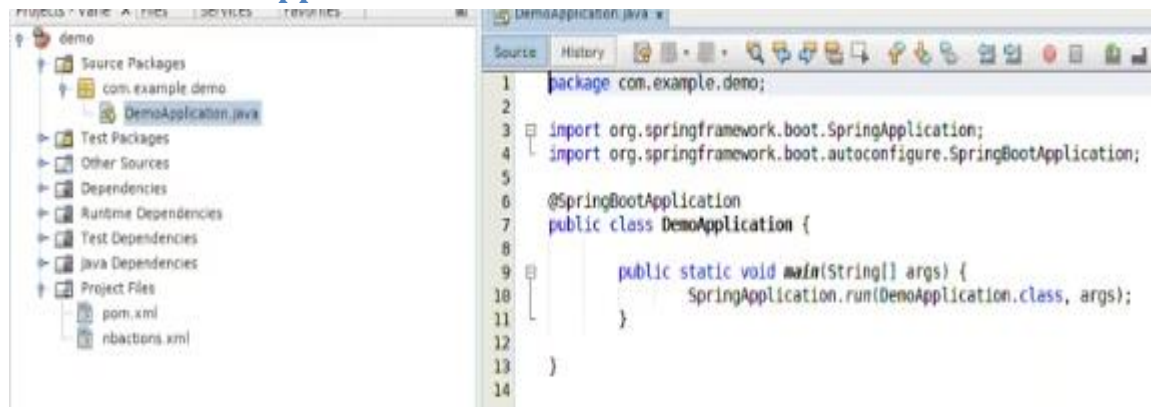
Pom.xml e le dipendenze di Spring

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

La nostra demoApplication



Non abbiamo caricato alcuna dipendenza e proveremo a fare qualcosa da riga di comando, per farlo possiamo implementare l'interfaccia `CommandLineRunner`;

Poi è sufficiente aggiungere l'annotazione alla classe `@Component` per dire a spring che deve istanziare la classe al posto nostro.

```
DemoApplication.java x CLI.java x
Source History
1 package com.example.demo;
2
3 import org.springframework.boot.CommandLineRunner;
4 import org.springframework.stereotype.Component;
5
6 @Component
7 public class CLI implements CommandLineRunner {
8
9     @Override
10    public void run(String... args) throws Exception {
11        if (args.length > 0) {
12            System.out.println(args[0]);
13        }
14    }
15
16 }
```

Ora per vedere come funziona la dependency injection di spring utilizziamo una nuova classe che è richiamata dalla CLI, la chiameremo SpringTransformer

SpringTransformer è il nostro Bean che deve essere automaticamente istanziato e per dire al framework di farlo va utilizzata l'annotazione **@Component**; invece per iniettare l'oggetto di tipo SpringTransformer nella classe CLI lo anoteremo con **@Autowired**

@Autowired

```
DemoApplication.java x CLI.java x StringTransformer.java x
Source History
1 package com.example.demo;
2
3 import org.springframework.stereotype.Component;
4
5 @Component
6 public class StringTransformer {
7
8     public String transform(String input) {
9         return input.toUpperCase();
10    }
11
12 }
```

```
DemoApplication.java x CLI.java x StringTransformer.java x
Source History
1 package com.example.demo;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.boot.CommandLineRunner;
5 import org.springframework.stereotype.Component;
6
7 @Component
8 public class CLI implements CommandLineRunner {
9
10    @Autowired
11    private StringTransformer stringTransformer;
12
13    @Override
14    public void run(String... args) throws Exception {
15        if (args.length > 0) {
16            System.out.println(stringTransformer.transform(args[0]));
17        }
18    }
19
20 }
```

Faccio un Maven Build in modo da creare la cartella target con dentro il jar del mio applicativo

Vado in console di windows (cmd), nel path target del mio progetto lancio il mio applicativo con un argomento

Java -jar target/demo-0.0.1-SNAPSHOT.jar ciao

Il risultato è CIAO

Potevamo avere lo stesso risultato in questo modo:

```
@Component
public class CLI implements CommandLineRunner {

    private final StringTransformer stringTransformer;

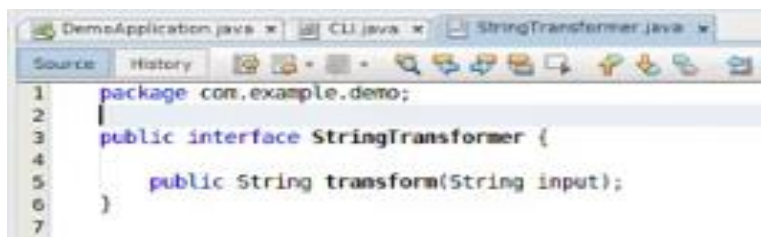
    @Autowired
    public CLI(StringTransformer stringTransformer) {
        this.stringTransformer = stringTransformer;
    }

    @Override
    public void run(String... args) throws Exception {
        if (args.length > 0) {
            System.out.println(stringTransformer.transform(args[0]));
        }
    }
}
```

Mettendo l'annotazione `@Autowired` nel costruttore la sostanziale differenza che la si ottiene sul field `StringTransformer` che potrà essere dichiarato come final.

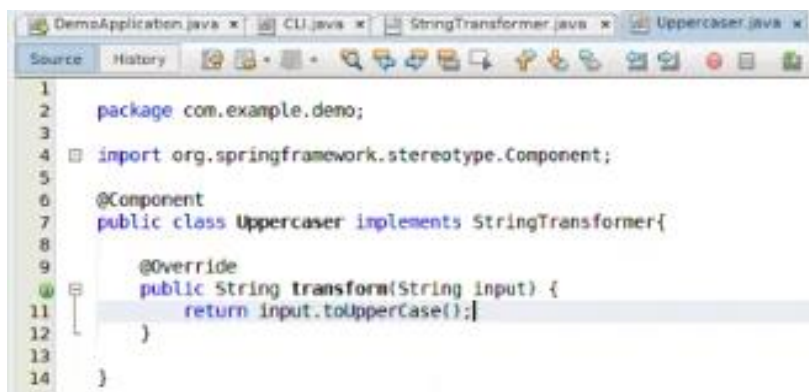
L'autowired può anche essere messo su un metodo setter ma è poco utilizzato.

Vediamo ora come è possibile gestire l'autowired su due istanze dello stesso tipo, cioè fare due classi che estendono o implementano `StringTransformer`.



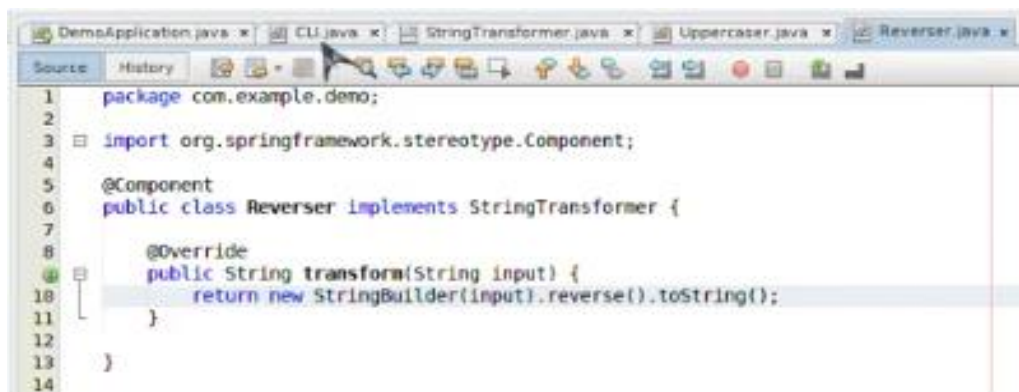
```
1 package com.example.demo;
2
3 public interface StringTransformer {
4
5     public String transform(String input);
6
7 }
```

SpringFramework dovrà istanziare le singole implementazioni di `StringTransformer`, perciò togliamo `@Component` per aggiungerlo qui:



```
1 package com.example.demo;
2
3
4 import org.springframework.stereotype.Component;
5
6 @Component
7 public class Uppercaser implements StringTransformer{
8
9     @Override
10     public String transform(String input) {
11         return input.toUpperCase();
12     }
13
14 }
```

E qui:



```
1 package com.example.demo;
2
3 import org.springframework.stereotype.Component;
4
5 @Component
6 public class Reverser implements StringTransformer {
7
8     @Override
9     public String transform(String input) {
10         return new StringBuilder(input).reverse().toString();
11     }
12
13 }
```

Ora da riga di comando possiamo richiamare i due bean distinti Uppercaser e Reverser

```
@Component
public class CLI implements CommandLineRunner {

    @Autowired
    private StringTransformer uppcaser;
    @Autowired
    private StringTransformer reverser;

    @Override
    public void run(String... args) throws Exception {
        if (args.length > 0) {
            System.out.println(uppcaser.transform(args[0]));
            System.out.println(reverser.transform(args[0]));
        }
    }
}
```

Se proviamo ad assegnare due nomi simili stringTransformer1 e stringTransformer2

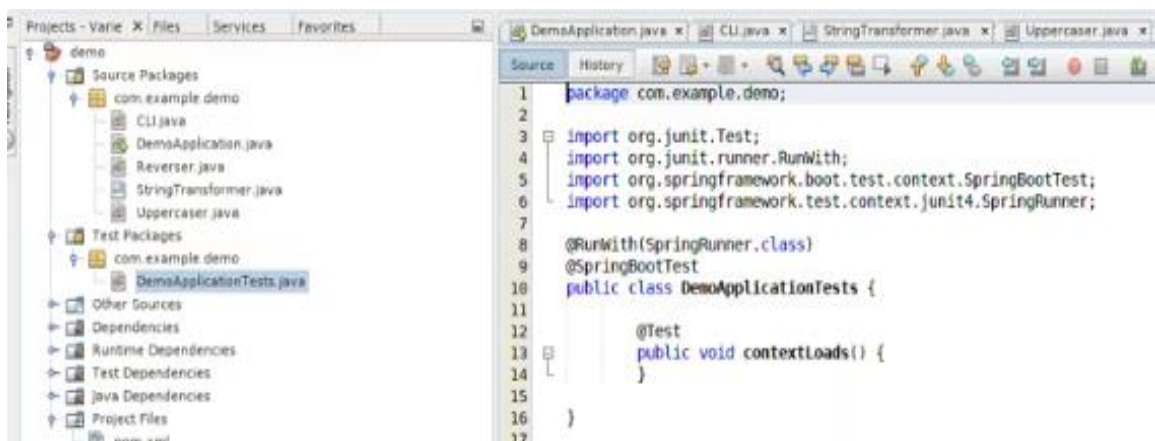
Fallisce durante la fase di test, come potete vedere qui:

```
*****
APPLICATION FAILED TO START
*****

Description:

Field transformer1 in com.example.demo.CLI required a single bean, but 2 were found:
- reverser: defined in file [/home/zonia/code/InitializrSpringbootProject/target/classes/com/example/demo/Reverser.class]
- uppcaser: defined in file [/home/zonia/code/InitializrSpringbootProject/target/classes/com/example/demo/Uppercaser.class]
```

Spring genera sempre classi di test



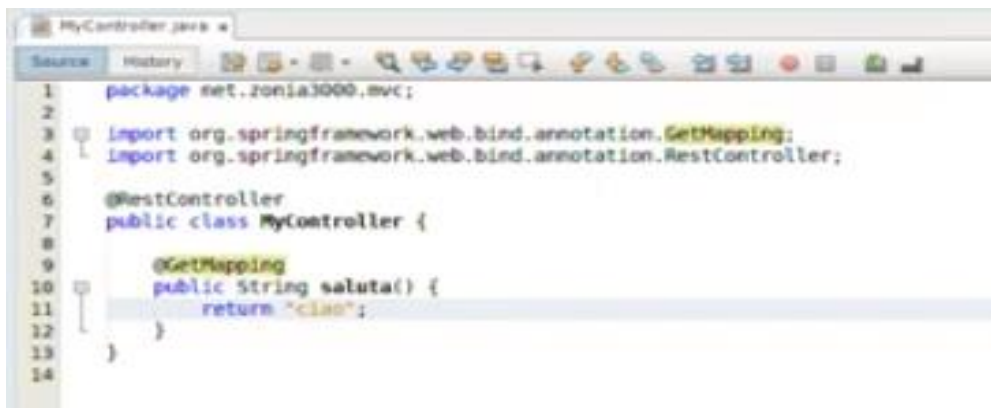
Per risolvere il problema bisogna qualificare i bean prima di @Component e prima @Autowired

Springboot per la realizzazione di un servizio Rest

Nella creazione del progetto bisogna includere le dipendenze web e devtools per non dover spegnere il server ogni volta che si fanno delle modifiche al codice.

Controller @RestController

Per gestire gli endpoint (chiamate a un url specifico) bisogna creare una classe Controller annotata con `@RestController` e gestire nel codice le richieste http(get e post).



```
1 package net.zonia3000.mvc;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RestController;
5
6 @RestController
7 public class MyController {
8
9     @GetMapping
10    public String saluta() {
11        return "ciao";
12    }
13 }
14
```

Annoteremo, quindi il metodo mappandolo come get `@GetMapping` in modo da gestire la richiesta in get e lanciando l'applicativo, verrà deployata sul server Tomcat e sarà visualizzata la risposta che è "ciao"

sulla radice di tomcat localhost:8080.

Se vogliamo dare un endpoint differente basterà annotarlo nel `@GetMapping ("/saluta")`

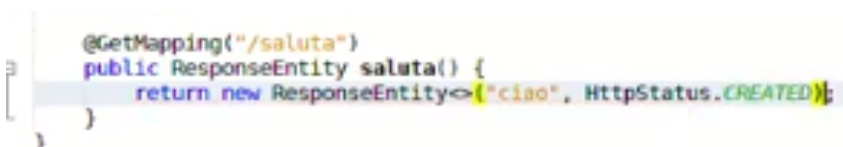
Sull'endpoint localhost:8080/saluta

Se vogliamo che raggruppare tutti gli endpoint della classe MyController bisogna mapparli tutti nella stessa richiesta **`@RequestMapping("/base")`**

L'endpoint diventerebbe localhost:8080/base/saluta

Lo status code

Lo status code dell'esempio precedente è 200, per modificare lo status code bisogna agire sulla risposta dandogli uno status code differente;



```
@GetMapping("/saluta")
public ResponseEntity saluta() {
    return new ResponseEntity<>("ciao", HttpStatus.CREATED);
}
```

La risposta non è più una stringa ma viene data una ResponseEntity con status code di tipo CREATED

@PathVariable per aggiungere un parametro nella risposta

Mappiamo il parametro nella richiesta in Get

```
@GetMapping("/saluta/{nome}")  
public ResponseEntity saluta(@PathVariable("nome") String nome) {  
    return new ResponseEntity("<iao " + nome, HttpStatus.CREATED);  
}
```

Localhost:8080/base/saluta/Pippo

Ciao Pippo

Altre informazioni recuperabili nella richiesta

https://docs.spring.io/spring/docs/current/spring-framework-reference/web.html		
java.io.Writer		
@PathVariable		For access to URI template variables. See URI patterns .
@MatrixVariable		For access to name-value pairs in URI path segments. See Matrix Variables .
@RequestParam		For access to the Servlet request parameters, including multipart files. Parameter values are converted to the declared method argument type. See @RequestParam as well as Multipart . Note that use of @RequestParam is optional for simple parameter values. See "Any other argument", at the end of this table.
@RequestHeader		For access to request headers. Header values are converted to the declared method argument type. See @RequestHeader .
@CookieValue		For access to cookies. Cookies values are converted to the declared method argument type. See @CookieValue .
@RequestBody		For access to the HTTP request body. Body content is converted to the declared method argument type by using <code>HttpMessageConverter</code> implementations. See @RequestBody .
HttpEntity		For access to request headers and body. The body is converted with an <code>HttpMessageConverter</code> . See HttpEntity .
@RequestPart		For access to a part in a multipart/form-data request, converting the part's body with an <code>HttpMessageConverter</code> . See Multipart .
java.util.Map, org.springframework.ui.Model, org.springframework.ui.ModelMap		For access to the model that is used in HTML controllers and exposed to templates as part of view rendering.
RedirectAttributes		Specify attributes to use in case of a redirect (that is, to be appended to the query string) and flash attributes to be stored temporarily until the request after redirect. See Redirect Attributes and Flash Attributes .

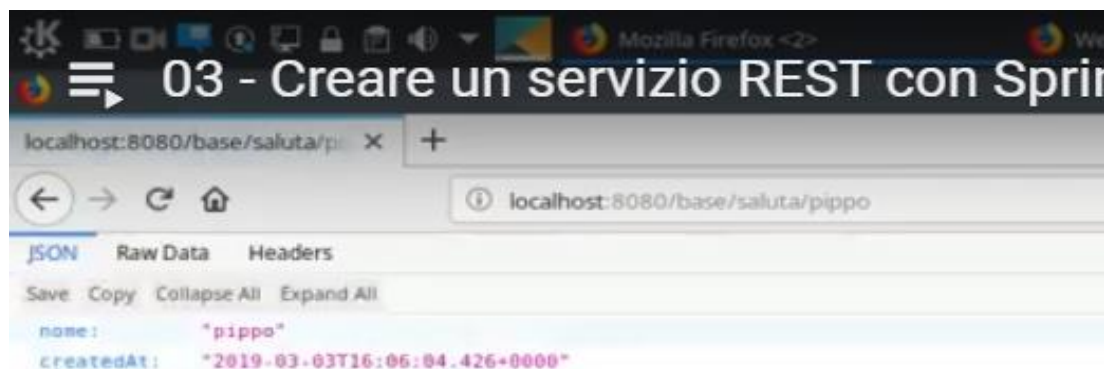
Conversione automatica di un oggetto in JSON

Creiamo una classe che faccia da modello dei nostri dati es. Utente

```
MyController.java | Utente.java
Source | History
1 package net.zonia3000.mvc;
2
3 import java.util.Date;
4
5 public class Utente {
6
7     private String nome;
8     private Date createdAt;
9
10    public Utente() {
11        createdAt = new Date();
12    }
13
14    public String getNome() {
15        return nome;
16    }
17
18    public void setNome(String nome) {
19        this.nome = nome;
20    }
21
22    public Date getCreatedAt() {
23        return createdAt;
24    }
25
26    public void setCreatedAt(Date createdAt) {
27        this.createdAt = createdAt;
28    }
29
30 }
```

```
@RestController
@RequestMapping("/base")
public class MyController {

    @GetMapping("/saluta/{nome}")
    public Utente saluta(@PathVariable("nome") String nome) {
        Utente u = new Utente();
        u.setNome(nome);
        return u;
    }
    //return new ResponseEntity<>("ciao " + nome, HttpStatus.CREATED);
}
```



Il risultato in JSON è automatico ma potete personalizzare sia l'input sia l'output per ottenere ad esempio un risultato in XML.

Una modifica interessante potrebbe essere ad esempio la modifica del formato della data, specificando nella classe utente il formato JSON che vogliamo.

```
@JsonFormat(pattern="yyyy-MM-dd")
```

```
Private Date createdAt;
```

Spring IOC

Ritorniamo a parlare dell'inversion of control per definire quelli che sono gli scoped dei Bean

Partiamo da un applicativo per capire questo concetto:

[Applicativo di riferimento ioc-applicationscope](#)

```
1 package demo.scope.beans;
2
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public abstract class AbstractBeanWithID {
6
7     private static final AtomicInteger COUNTER = new AtomicInteger();
8
9     private final int id;
10
11     public AbstractBeanWithID() {
12         id = COUNTER.addAndGet(1);
13     }
14
15     public int getId() {
16         return id;
17     }
18 }
19
```

Questa classe astratta estende i nostri bean e in base allo scoped di ciascuno di loro capiamo quante volte viene istanziata incrementando un contatore di tipo atomico per il thread safe...

```
1 package demo.scope.beans;
2
3 import org.springframework.stereotype.Component;
4
5 @Component
6 public class DefaultBean extends AbstractBeanWithID {
7 }
8
```

Primo bean senza alcuno scoped specificato.

Avrà lo scoped di default che per Spring è il singleton (una sola istanza)

```
1 package demo.scope.beans;
2
3 import org.springframework.stereotype.Component;
4 import org.springframework.web.context.annotation.RequestScope;
5
6 @Component
7 @RequestScope
8 public class RequestBean extends AbstractBeanWithID {
9 }
10
```

Questo bean annotato con @RequestScope viene creato una nuova istanza per ogni richiesta ricevuta.

```
1 package demo.scope.beans;
2
3 import org.springframework.stereotype.Component;
4 import org.springframework.web.context.annotation.SessionScope;
5
6 @Component
7 @SessionScope
8 public class SessionBean extends AbstractBeanWithID {
9 }
10
```

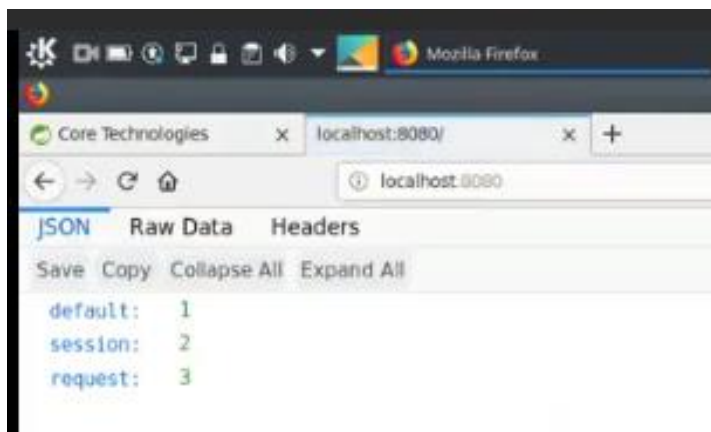
Questo bean annotato con @SessionScope viene creato una nuova istanza per ogni sessione

```

1 package demo.scope;
2
3 import demo.scope.beans.*;
4 import java.util.LinkedHashMap;
5 import java.util.Map;
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.web.bind.annotation.GetMapping;
8 import org.springframework.web.bind.annotation.RestController;
9
10 @RestController
11 public class MyController { ← uno
12
13     @Autowired
14     private DefaultBean defaultBean;
15
16     @Autowired
17     private SessionBean sessionBean;
18
19     @Autowired
20     private RequestBean requestBean; ← tanti
21
22     @GetMapping("/")
23     public Map<String, Integer> showBeans() {
24
25         Map<String, Integer> beans = new LinkedHashMap<>();
26
27         beans.put("default", defaultBean.getId());
28         beans.put("session", sessionBean.getId());
29         beans.put("request", requestBean.getId());
30
31         return beans;
32     }
33 }
34

```

Nel controller m'inietto i tre bean e mi faccio restituire l'id che viene creato incrementando il contatore, all'atto dell'esecuzione dell'applicativo verificherò se è cambiato l'id e quindi se è incrementato il contatore di creazione di una nuova istanza.



Se aggiorno la pagina l'unico id che s'incrementa è quel request, per vedere l'incremento dell'id di sessione basta caricare una nuova pagina in incognito.

```
private final RequestBean requestBean;
```

```
@Autowired
```

```
public MyController(RequestBean requestBean) {
    this.requestBean = requestBean;
}
```

Questo funziona perché vengono usati i proxy, che vengono costruiti automaticamente dal framework. Infatti se stampiamo la classe di defaultBean, sessionBean e requestBean

```
System.out.println(defaultBean.getClass());
```

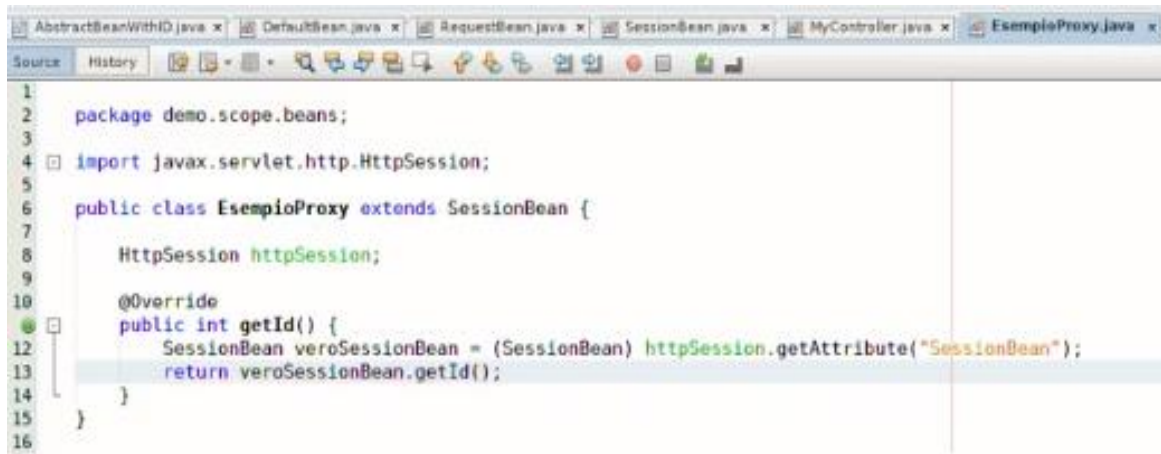
```

2019-03-19 20:21:39.939 INFO 8118 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet :
2019-03-19 20:21:59.951 INFO 8118 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet :
class demo.scope.beans.DefaultBean
class demo.scope.beans.SessionBean$$EnhancerBySpringCGLIB$$60fc9f43
class demo.scope.beans.RequestBean$$EnhancerBySpringCGLIB$$11c4c2fc

```

Solo per la DefaultBean la classe resta la stessa.

Cosa fa il proxy



```

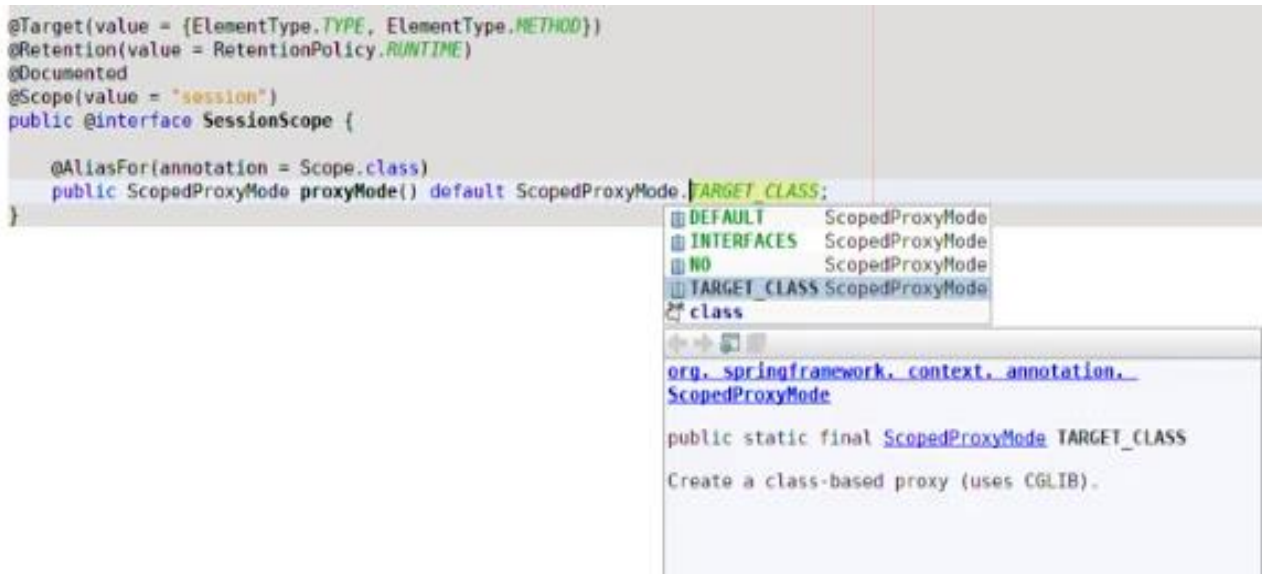
1 package demo.scope.beans;
2
3 import javax.servlet.http.HttpSession;
4
5 public class EsemplioProxy extends SessionBean {
6     HttpSession httpSession;
7
8     @Override
9     public int getId() {
10         SessionBean veroSessionBean = (SessionBean) httpSession.getAttribute("SessionBean");
11         return veroSessionBean.getId();
12     }
13 }
14
15
16

```

Il proxy è una classe che estende il bean nella quale viene fatto l'override dei metodi

Nel nostro caso del metodo getId(), recupera il vero oggetto SessionBean partendo dalla httpSession e ne restituisce il vero getId();

La nostra classe SessionBean viene modificata



```

@Target(value = {ElementType.TYPE, ElementType.METHOD})
@Retention(value = RetentionPolicy.RUNTIME)
@Documented
@Scope(value = "session")
public @interface SessionScope {

    @AliasFor(annotation = Scope.class)
    public ScopedProxyMode proxyMode() default ScopedProxyMode.TARGET_CLASS;
}

```

DEFAULT ScopedProxyMode
 INTERFACES ScopedProxyMode
 NO ScopedProxyMode
 TARGET_CLASS ScopedProxyMode
 class

org.springframework.context.annotation.ScopedProxyMode
 public static final ScopedProxyMode TARGET_CLASS
 Create a class-based proxy (uses CGLIB).

PrototypeBean

```
package demo.scope.beans;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import org.springframework.web.context.annotation.RequestScope;

@Component
@RequestScope
public class RequestBean extends AbstractBeanWithID {

    @Autowired
    private PrototypeBean prototypeBean;

    public PrototypeBean getPrototypeBean() {
        return prototypeBean;
    }

}
```

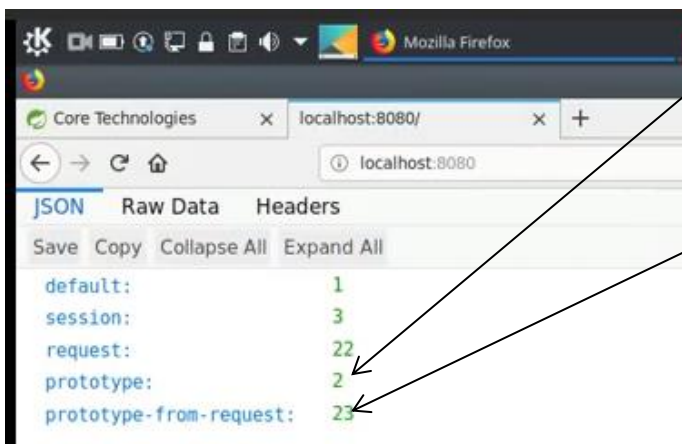
Adesso andiamo ad aggiungere nel MyController il Bean prototype

MyController → @Autowired

```
private PrototypeBean prototypeBean;
```

```
beans.put("prototype", prototypeBean.getId());
```

```
beans.put("prototype-from-request", requestBean.getPrototypeBean().getId());
```



IL prototype all'interno di MyController non cambia.

Il prototyper generato dentro la richiesta invece cambia a ogni richiesta.

Se definite il proxymode però il comportamento è diverso.

```
@Component
@Scope(value = BeanDefinition.SCOPE_PROTOTYPE, proxyMode = ScopedProxyMode.TARGET_CLASS)
public class PrototypeBean extends AbstractBeanWithID {

}
```

L'id cambia sia al prototype che al prototype-from-request

proxyMode di tipo Interfaces

```
@Component
@Scope(value = BeanDefinition.SCOPE_PROTOTYPE, proxyMode = ScopedProxyMode.INTERFACES)
public class PrototypeBean extends AbstractBeanWithID {

}
```

Ritorna un errore e questo perché tramite Autowired stiamo richiedendo una classe e non un'interfaccia

```
Error starting ApplicationContext. To display the conditions report re-run your application with 'debug' enabled.
2019-03-19 20:34:47.662 ERROR 8118 --- [ restartedMain] o.s.b.d.LoggingFailureAnalysisReporter :

*****
APPLICATION FAILED TO START
*****

Description:

The bean 'prototypeBean' could not be injected as a 'demo.scope.beans.PrototypeBean' because it is a JDK dynamic proxy that implements:
```

Proviamo a creare un'interfaccia e la facciamo implementare dalla classe astratta che estendeva i Beans.

```
public interface IdHolder {

    int getId();

}

public abstract class AbstractBeanWithID implements IdHolder {
```

A quel punto getId() diventa un metodo implementato dell'interfaccia

```
@Component
@RequestScope
public class RequestBean extends AbstractBeanWithID {

    @Autowired
    private IdHolder prototypeBean;

    public IdHolder getPrototypeBean() {
        return prototypeBean;
    }
}
```

Questo occorre per andare a fare il prototype del requestBean

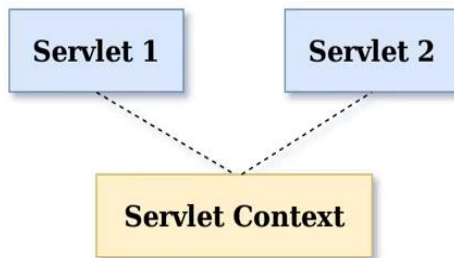
Dati alla mano è sempre conveniente usare TARGET_CLASS

Bean scopes

Singleton vs Application

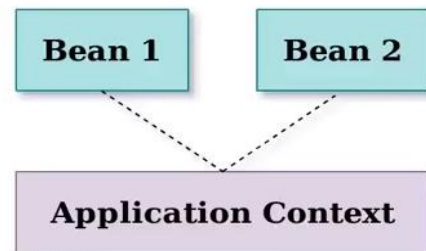
Vedendo la documentazione , ci dice questo:

- Il singleton crea un'unica istanza per ogni Spring IoC container
- L'application crea un'istanza per ogni servletContext



Java EE

Il servlet context è una classe che è condivisa da tutte le servlet che stanno all'interno dello stesso container. (tutte le servlet che girano



Spring

L'application context è una classe specifica di Spring che rappresenta un IoC container.

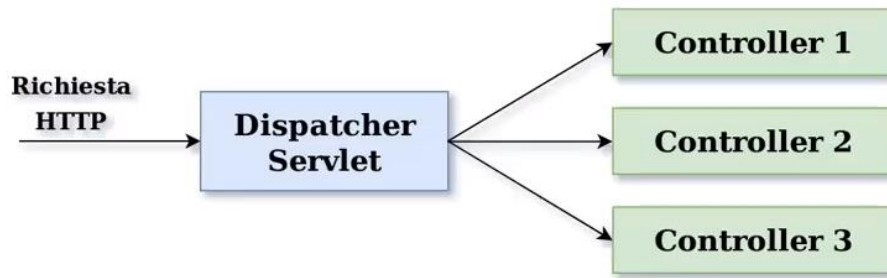
Il metodo run di SpringBoot genera la nostro application Context

```
@SpringBootApplication
public class IoScopeApplication {

    public static void main(String[] args) {
        SpringApplication.run(IoScopeApplication.class, args);
    }
}
```

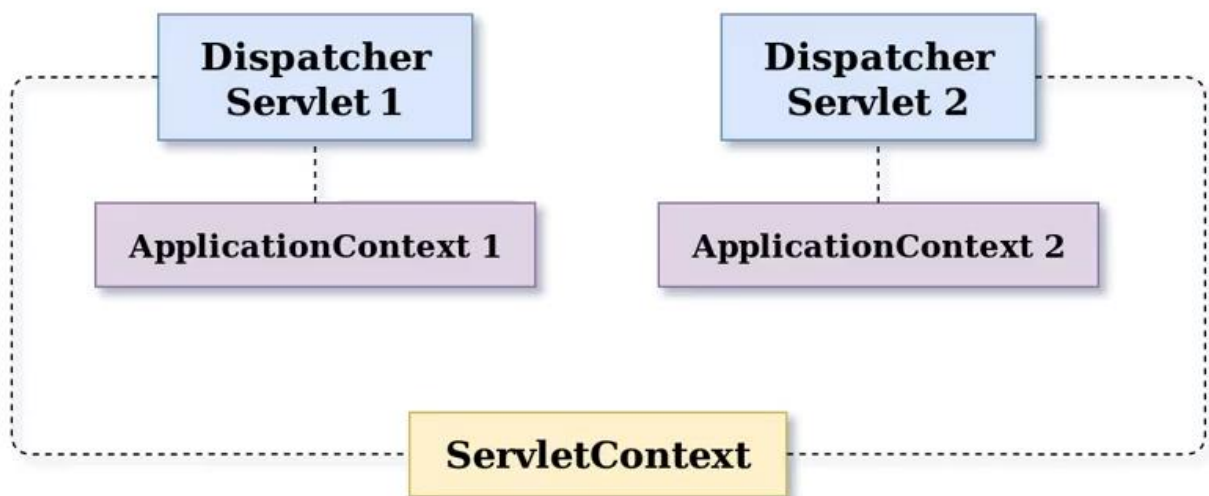
The image shows a code editor with the above Java code. A tooltip is visible over the `run` method call, showing two overloads: `run(Class<?> primarySource, String... args) ConfigurableApplicationContext` and `run(Class<?>[] primarySources, String[] args) ConfigurableApplicationContext`. A mouse cursor is pointing at the second overload.

Ogni volta che inizializzate Spring MVC, invece, viene creata una Dispatcher Servlet



Che si occupa di smistare tutte le richieste http in arrivo al controller opportuno.

Ad ogni Dispatcher Servlet potete associare un'ApplicationContext e quindi un IoC Container.



Se Avete più Dispatcher Servlet è come avere più IoC Container all'interno dello stesso ServletContext

Questa è una possibilità ma in realtà ad una servletContext di solito viene associata una sola ApplicationContext e quindi sono la stessa cosa.

Insieme dei progetti a cui faccio riferimento nelle esercitazioni di Springboot

Esercitazioni di Spring Boot 1: anatomia di un progetto

Qui analizzeremo più in dettaglio la struttura del progetto e i file da cui è costituito, creando un primo esempio di controller REST.

File pom.xml

Nel file di pom diciamo a Maven quali sono le dipendenze Spring necessarie al progetto; partendo da una base di progetto Spring; quindi bisogna ereditare dal pom di Spring la configurazione principale di Spring

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.2.1.RELEASE</version>
  <relativePath /> <!-- lookup parent from repository -->
</parent>
```

Poi a seconda delle necessità aggiungere la parte di configurazione necessaria a identificare la dipendenza, la dipendenza viene recuperata tramite la coordinata del progetto

- Coordinate del progetto
- GroupId → percorso PATH in cui è salvato il progetto
- ArtifactId → il nome del progetto
- Versione → versione del progetto
- Scope → di default è compile, quando non è specificato, sarà compile e le classi del progetto saranno disponibili già in fase di compilazione.

Classe Main che equivale alla Entrypoint del progetto

E' La classe principale, rappresenta l'entry point della nostra applicazione e appunto contiene il metodo **main()** all'interno del quale è chiamato il metodo statico **SpringApplication.run()**.

```
• @SpringBootApplication
• public class entrypointApplication {
•
•     public static void main(String[] args) {
•         SpringApplication.run(entrypointApplication.class, args);
•     }
• }
```

Conoscenza di alcune annotazione di Spring

@Configuration consente alle classi che ne fanno uso di definire dei metodi per la creazione di oggetti (bean) il cui ciclo di vita sarà gestito direttamente da Spring. Questo è

particolarmente utile quando si utilizzano librerie di terze parti di cui non si dispone o non si vuole utilizzare il codice sorgente.

Utilizzo di un file.xml di configurazione o di una classe di configurazione

Config.java o config.xml

Anziché lasciare completamente a Spring il compito di istanziare gli oggetti, in questo modo decidiamo noi come costruire il bean:

```
• public class Bean1 {  
•     private final String message;  
•  
•     public String getMessage() {  
•         return message;  
•     }  
• }  
•  
• Config.java → riferito al Bean1 che puoi ben vedere non ha @Component e quindi dobbiamo dire  
• noi come deve essere istanziata.  
• @Configuration  
• @ImportResource("classpath:/config.xml") // (src/main/resources/config.xml)  
• public class Config {  
•  
•     @Bean  
•     public Bean1 getBean2() {  
•         return new Bean1();  
•     }  
• }
```

N.B. che qualora tu non specifichi il valore per il nome della classe all'atto dell'istanziazione @Component(value="nomeOggetto"); sarà presa come nome istanza il nome della classe con la prima lettera minuscola; nel nostro caso Bean1, bean1.

```
• public class Bean2 {  
•     private final String message;  
•  
•     public Bean2(String message) {  
•         this.message = message;  
•     }  
•     public String getMessage() {  
•         return message;  
•     }  
• }  
•
```

- Config.xml → Questo è un modo alternativo di configurare un bean2, anch'esso non annotato come @Component
- `<?xml version="1.0" encoding="UTF-8"?>`
- `<beans xmlns="http://www.springframework.org/schema/beans"`
- `xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"`
- `xsi:schemaLocation="http://www.springframework.org/schema/beans`
- `http://www.springframework.org/schema/beans/spring-beans.xsd">`
- `<!-- Questo tipo di configurazione si usava soprattutto in passato -->`
- `<bean id="bean2" class="test.Bean2">`
- `<constructor-arg value="Ciao da Bean2" />`
- `</bean>`
- `</beans>`

@EnableAutoConfiguration attiva il meccanismo di configurazione automatica dei bean che saranno gestiti direttamente dal framework.

@ComponentScan consente a Spring di ricercare tutte le classi annotate con **@Component** o con una sua specializzazione come **@Controller**, **@RestController**, **@Repository**, ecc. Da notare che di default la scansione riguarda solo il package in cui è inclusa l'applicazione principale e i suoi sottopackage.

Classe ServletInitializer

Questa classe, che tralascieremo per ora, è utile per configurare opportunamente la servlet che gestirà le richieste provenienti dai client.

Application.properties

File Collocato nella cartella resources è utilizzato per memorizzare proprietà e configurazioni per connettersi ad un database, cambiare la porta del server embedded o utilizzare correttamente altre risorse. application.properties.

Esercitazioni di Spring Boot 2: il nostro primo controller REST

L'idea è di creare una semplice API basata su **REST** che ci consenta di interrogare un database di calciatori e di fare le classiche operazioni CRUD.

Chiamerò il mio progetto **Players** per cui Spring Initializer genererà in automatico la classe **PlayersApplication**.

Modello dati

Dobbiamo invece creare la classe **Player** che modella il nostro calciatore. Sono sufficienti pochi attributi come l'id (chiave primaria di tipo integer), il nome, il cognome e la squadra di appartenenza (tutti di tipo String) con relativi getter e setter.

la classe **PlayersController** (**@RestController**)

E' una classe annotata con **@RestController**, questa annotazione serve propria a dire a Springframework di prendere in carico questa classe per la costruzione del servizio Rest.

Sarà quindi il framwork ad istanziare questa classe all'avvio dell'applicazione, rendendo disponibili i servizi da essa offerti. (I classici servizi della Classe Servlet o sue derivate `HttpServletRequest` ad esempio).

EndPoint **"/players"**

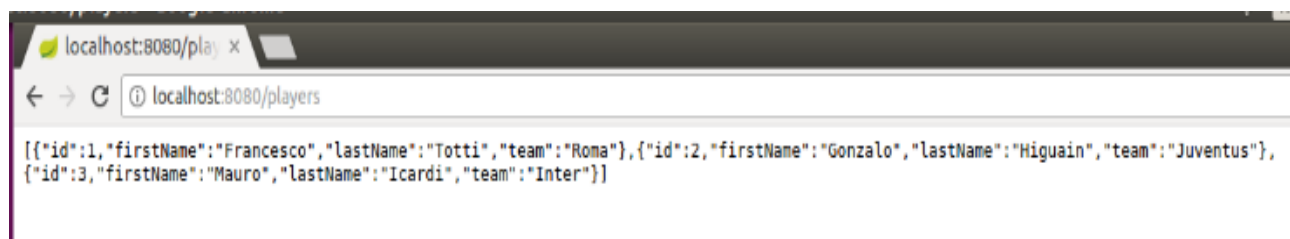
Il servizio offerto da questo progetto è un singolo endpoint **"/players"** che restituisce un elenco di calciatori. Banalmente un metodo pubblico **getAllPlayers()**, che ritorna una lista i cui elementi sono di tipo **Player**.

@RequestMapping, **@GetMapping**

Dichiarano endpoint, nel nostro caso con nome **"/players"** che ne indica il path finale dell'Url

Una **@RequestMapping** implicitamente è una **@GetMapping**, ma vi suggerisco di dichiararla direttamente **@GetMapping**

Il risultato o quello che vedremo nel browser sarà la lista di calciatori descritta in JSON.



Springboot : gestione della logica di business

La logica di business è la parte di codice del nostro applicativo che si preoccupa di far interagire l'interfaccia utente (logica di presentazione) con l'elaborazione intermedia dei dati, prima della fase di persistenza sul database.

Un service, è un servizio di logica di business, e quindi un controller che fa da tramite tra la logica di business e logica di presentazione. Nel controller vengono separati i metodi che gestiscono effettivamente le richieste (i metodi equivalgono gli endpoint del nostro url)

Ad esempio potrebbe essere necessario avere nel controller con metodo endpoint per aggiungere un nuovo record, uno per aggiornarne i dati ed un altro ancora per cancellare un record dalla lista.

```
• import java.util.ArrayList;
• import java.util.Arrays;
• import java.util.List;
•
• import org.springframework.stereotype.Service;
•
• @Service
• public class PlayerService {
•
•     private List<Player> players = new ArrayList<> (Arrays.asList(new Player(1, "Francesco",
•         "Totti", "Roma", "midfielder"),
•
•         new Player(2, "Gonzalo", "Higuain", "Juventus", "forward"), new Player(3,
•         "Mauro", "Icardi", "Inter", "forward")));
•
•     public List<Player> getAllPlayers() {
•         return players;
•     }
•
•     public Player getPlayer(int id) {
•
•         return players.stream()
•             .filter(p -> p.getId() == id)
•             .findFirst()
•             .get();
•     }
•
•     public void addPlayer(Player player){
•         players.add(player);
•     }
•
•     public void updatePlayer(int id, Player player) {
•         for(int i=0; i < players.size(); i++){
•             Player p = players.get(i);
•             if(p.getId()==id){
•                 players.set(i, player);
•             }
•         }
•     }
• }
```

```

•
•
•
• public void deletePlayer(int id) {
•     players.removeIf(p -> p.getId() == id);
•
• }
•
•
• }

```

Niente di più semplice. Dal punto di vista di Spring va rilevata solo la presenza dell'annotazione **@Service** per denotare la tipologia della classe e consentire al framework di gestirne il lifecycle.

controller REST

Con l'annotazione **@RequestMapping** possiamo marcare gli endpoint del controller specificando il path di riferimento ed il tipo di richiesta (GET, POST, PUT, DELETE). In questo modo la DispatcherServlet di Spring smisterà correttamente le richieste HTTP provenienti dai client.

Per le GET e POST esistono specifiche annotazioni quali **@GetMapping** e **@PostMapping**.

```

• @GetMapping("/players")
•
• equivale a
•
• @RequestMapping(value = "/players", method = RequestMethod.GET)
• -----
• @PostMapping("/players")
•
• Equivale a
•
• @RequestMapping(value = "/players", method = RequestMethod.POST)

```

Si osservi che in entrambi i casi abbiamo lo stesso endpoint **“/players”** ma, essendo differente la tipologia di richiesta (in un caso GET e nell'altro POST), non ci sarà alcun fraintendimento per il controller.

```

@RestController
public class PlayersController {

    @Autowired
    private PlayerService playerService;

    @GetMapping("/players")
    public List<Player> getAllPlayers() {
        return playerService.getAllPlayers();
    }
}

```

```

@GetMapping("/players/{id}")
public Player getPlayer(@PathVariable int id) {
    return playerService.getPlayer(id);
}

@PostMapping("/players")
//Alternative
//@RequestMapping(value = "/players", method = RequestMethod.POST)
public void addPlayer(@RequestBody Player player){
    playerService.addPlayer(player);
}

@RequestMapping(value = "/players/{id}", method = RequestMethod.PUT)
public void updatePlayer(@PathVariable int id, @RequestBody Player
player){
    playerService.updatePlayer(id, player);
}
@RequestMapping(value = "/players/{id}", method = RequestMethod.DELETE)
public void deletePlayer(@PathVariable int id){
    playerService.deletePlayer(id);
}
}

```

Inniettiamo con Spring Autowired la classe @Service che la classe intermedia tra il nostro oggetto Player e la tabella. Ricapitolando, il controller prende un servizio @Service (Players) e crea gli endpoint necessari all'utente per interagire con il database.

- @Autowired
- private PlayerService playerService;

in modo da richiamarne i metodi del @service direttamente nei vari endpoint del controller.

add

Ecco come viene gestita l'aggiunta di un nuovo calciatore.

- PostMapping("/players")
public void addPlayer(@RequestBody Player player){
playerService.addPlayer(player);

Ho voluto soffermarmi sull'aggiunta perché volevo farvi notare che nonostante il parametro sia annotato come @RequestBody; per il metodo playerService non è un problema perché sarà spring a preoccuparsi di estrarre dalla richiesta in JSON l'effettivo Oggetto POJO da aggiungere nel servizio.

Update → richiesta in PUT

- @RequestMapping(value = "/players/{id}", method = RequestMethod.PUT)
- public void updatePlayer(@PathVariable int id, @RequestBody Player player){

```
• playerService.updatePlayer(id, player);  
• }
```

Delete → richiesta in delete

```
• @RequestMapping(value = "/players/{id}", method = RequestMethod.DELETE)  
• public void deletePlayer(@PathVariable int id){  
•     playerService.deletePlayer(id);  
• }
```

N.B. Sia nell'update che nel delete l'annotazione per parametro è **@PathVariable** il parametro prendere il valore direttamente dall'endpoint **"/players/{id}"**

Spring Boot, come lavorare con i database

Intruduzione

In questa nuova esercitazione vedremo come collegare il nostro servizio REST ad un database MySQL in cui verranno memorizzati i dati relativi ai calciatori.

Per fare questo ci serviremo del progetto “[Spring Data](#)” che fornisce tutti gli strumenti necessari per gestire la persistenza. In particolare offre delle interfacce che consentono l’accesso ai dati, indipendentemente dal DBMS utilizzato.

Dipendenza al JPA

In primo luogo sono richieste due dipendenze per gestire il **JPA (Java Persistence API)**

La dipendenza spring-data con la quale spring si occupa in maniera autonoma della persistenza dei dati sul db

```
• <dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
• </dependency>
```

e MySQL (è un opportuno connectore per interfacciare JDBC al database MySQL)

```
• <dependency>
• <groupId>mysql</groupId>
• <artifactId>mysql-connector-java</artifactId>
• </dependency>
```

Application.properties parametri di connessione al database

Nel file “*application.properties*” vanno aggiunti i parametri di configurazione del database utilizzato.

```
• spring.jpa.hibernate.ddl-auto=create
• spring.datasource.url=jdbc:mysql://localhost:3306/nome del database
• spring.datasource.username=root
• spring.datasource.password=root
```

Una particolare attenzione va riservata alla proprietà specificata nella prima riga, con la quale crea in automatico lo schema della tabella ad ogni esecuzione dell’applicativo. Il suo utilizzo “**in produzione**” è caldamente sconsigliato perché, lavorando con un database esistente, si andrebbero a cancellare i dati già presenti.

SSL connection

Un'altra particolarità è SSL connection, che di default dovrebbe essere non attiva, ma non sempre è così, perciò consiglio di disabilitarlo nella stringa di connessione:

- `spring.datasource.url=jdbc:mysql://localhost:3306/nome del database?useSSL=false`

Entità (POJO)

Una classe, opportunamente annotata, dove ogni entità farà da mapping ai dati contenuti in una tabella nel nostro database.

@Entity indica per l'appunto che la classe rappresenta un'entità e come tale verrà trattata da Spring, mentre @id identifica l'attributo che fungerà da chiave primaria per il database. In particolare abbiamo scelto la generazione automatica dei valori. Volendo sarebbe possibile assegnare direttamente un valore all'id ma occorrerebbe gestire i vincoli imposti dalla primary key. Se non strettamente necessario meglio lasciar fare al framework.

Per una migliore organizzazione del codice tutte le classi entity vengono create nel package *models*,

Interazione con il database

L'interazione con il database avviene tramite repositories, ovvero tramite un'interfaccia @repository (specializzazione della @Component) che estendendo una JpaRepository, questa estensione mette a disposizione di suo una serie di metodi utili alla gestione del database stesso (findAll(), findOne(), save(), delete(), query_method())

- @Repository
public interface (nomedatabase)Repository extends JpaRepository<nomedatabase Long> {
}
- Nel caso specifico del progetto
- @Repository
public interface PlayerRepository extends JpaRepository<Player, Long> {
}

Adesso l'unica cosa da fare è collegare il **repository** al **controller REST**, attraverso un **service**.

Nella [precedente esercitazione](#) abbiamo parlato dell'importanza di un service per separare gli endpoint presenti nel controller dai metodi che gestiscono effettivamente le richieste.

[PlayerServiceDB](#). sarà il nostro repository riferito al database che il controller REST .deve conoscere per recuperare i dati direttamente dal database

Quindi adesso anziché iniettare la nostra vecchia classe playerService che faceva da servizio per i dati, adesso il service dei dati ce lo fornirà il DB.

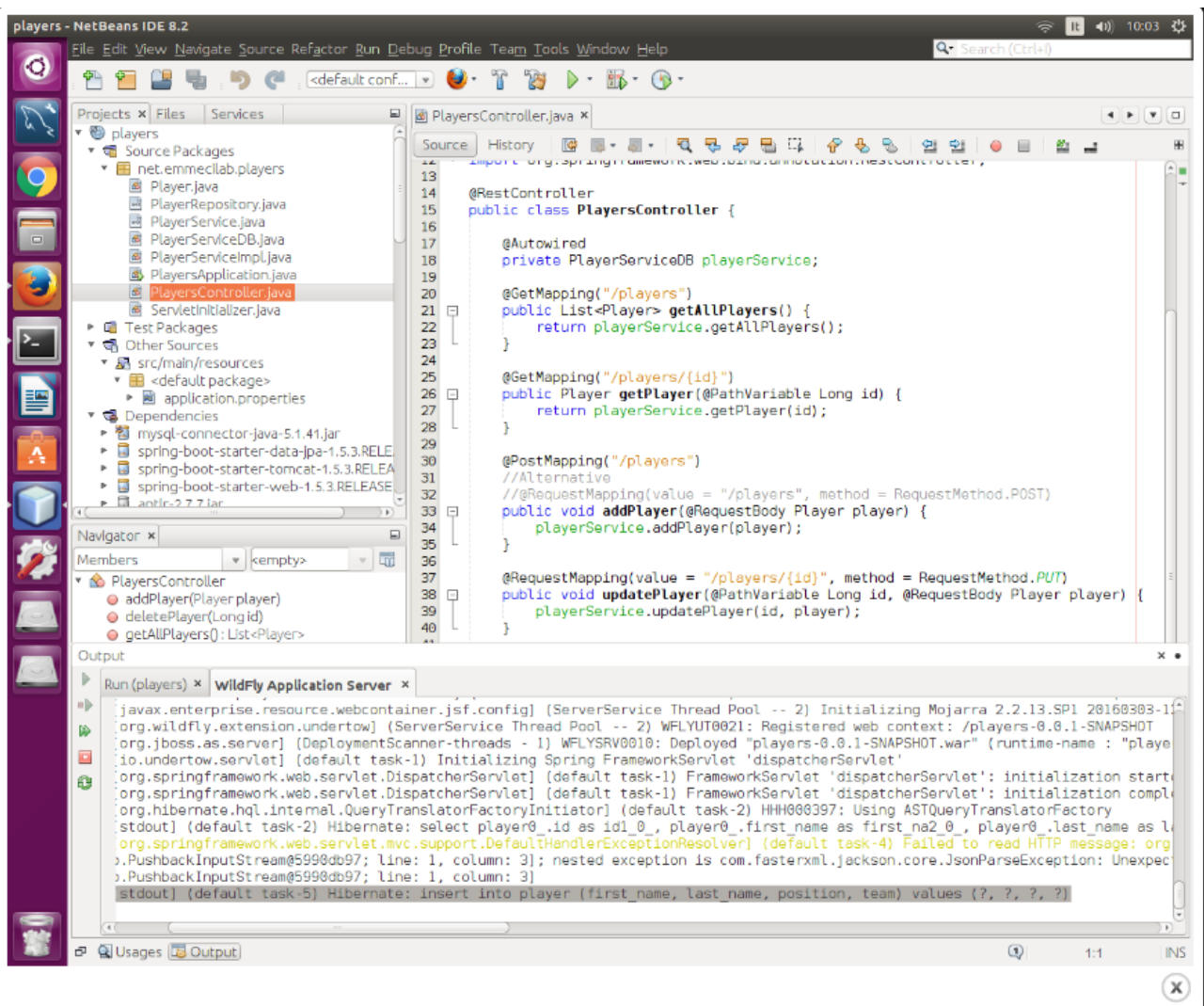
- `@Autowired`
`private PlayerServiceDB playerService;`

Inserimento di un calciatore da CURL

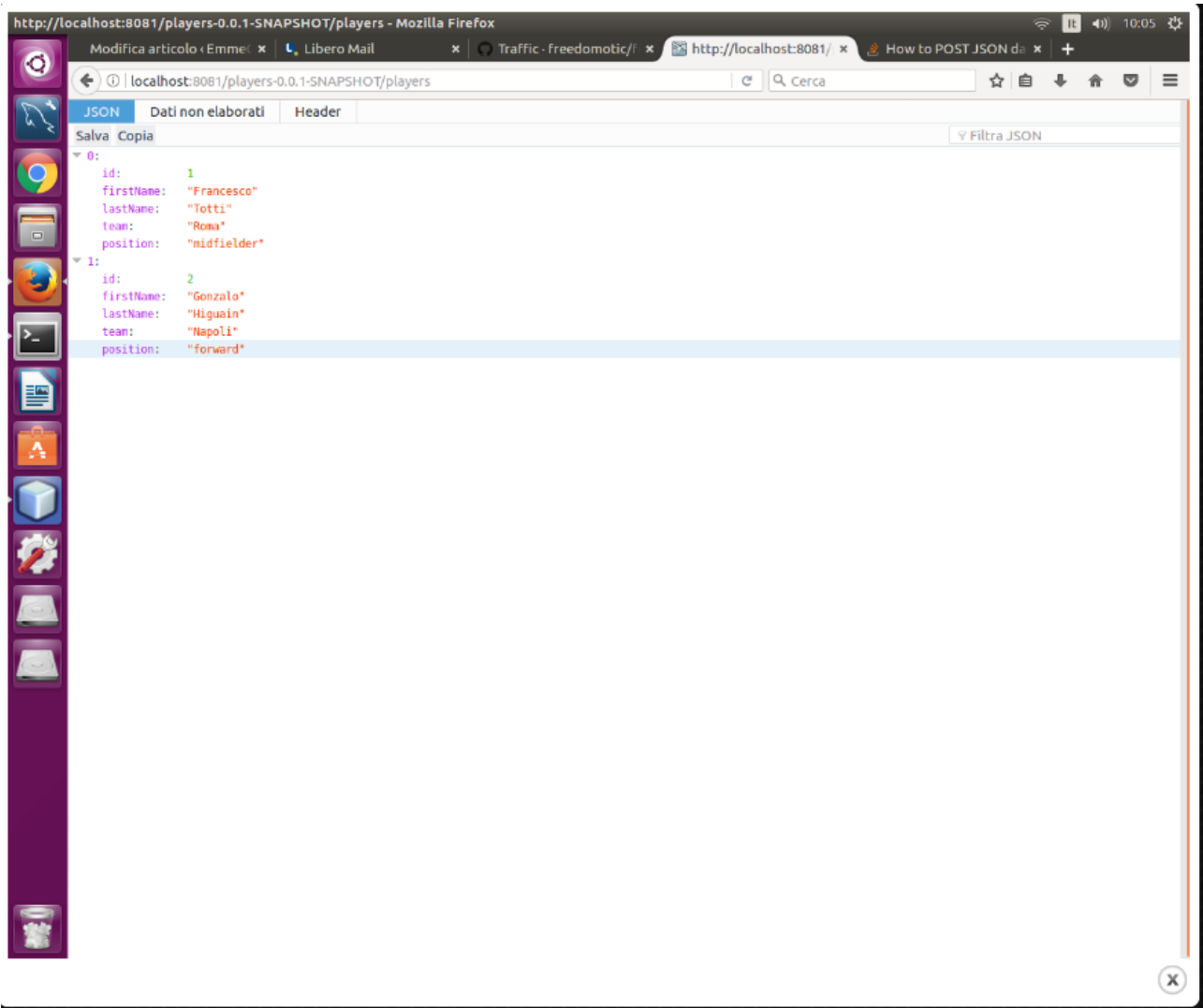
Dopo aver compilato ed eseguito l'applicazione, proviamo ad inserire un nuovo giocatore da riga di comando

- `curl -X POST -H "Content-Type: application/json" -d
'{"firstName":"Gonzalo","lastName":"Higuain","team":"Napoli","position":"forward"}'
http://localhost:8081/players-0.0.1-SNAPSHOT/players`

Come possiamo notare dai log l'inserimento è avvenuto con successo



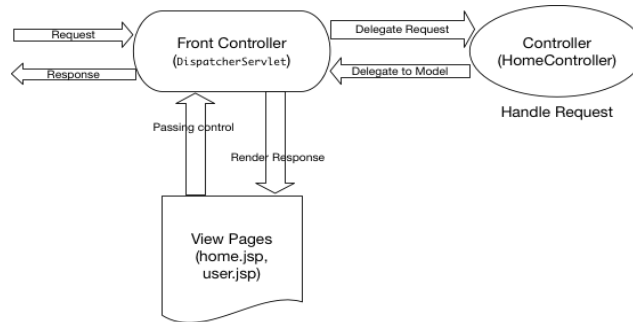
Aggiornando la lista dei calciatori vedremo un nuovo elemento a cui è stata assegnata automaticamente la primary key che non avevamo specificato nel json.



Creare un Dynamic Web Project con l'aggiunta di Spring

Architettura del progetto

Spring MVC si basa sull'architettura Model-View-Controller. Qui sotto vi mostro come si presenta il progetto che faremo con questo tipo di architettura.



Una Url sul browser produce una richiesta che viene inizialmente presa in carico dal DispatcherServlet, ne abbiamo già parlato [qui](#).

La dispatcherServlet delega ad un opportuno controller la richiesta ricevuta ed invia la risposta ad una view, (pagina xhtml, html o jsp per impaginarla).

- Automaticamente Spring configura implicitamente una sua dispatcher Servlet per ascoltare le richieste dei controller che stanno nello stesso package o quelli inferiori rispetto alla classe che contiene lo SpringApplication → `SpringApplication.run(ClasseCheContieneIlMain.class, args);`

Ora siamo noi a dichiarare esplicitamente la nostra dispatcherServlet nel file web.xml

dispatcherServlet nel file web.xml

contextConfigLocation init-param fornisce il percorso del file spring-servlet che è il file di configurazione dei beans.

In questo file vengono risolti tutti i problemi di comunicazione tra la DispatcherServlet con i controller e le views.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://java.sun.com/xml/ns/javaee" xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd" id="WebApp_ID">
  <display-name>spring-mvc-example</display-name>

  <!-- Add Spring MVC DispatcherServlet as front controller -->
  <servlet>
    <servlet-name>spring</servlet-name>
    <servlet-class>
      org.springframework.web.servlet.DispatcherServlet
    </servlet-class>
    <init-param>
      <param-name>contextConfigLocation</param-name>
      <param-value>/WEB-INF/spring-servlet.xml</param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
  </servlet>

  <servlet-mapping>
    <servlet-name>spring</servlet-name>
    <url-pattern>/</url-pattern>
  </servlet-mapping>
</web-app>
```

Spring beans configuration

```
<?xml version="1.0" encoding="UTF-8"?>
<beans:beans xmlns="http://www.springframework.org/schema/mvc"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:beans="http://www.springframework.org/schema/beans"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="http://www.springframework.org/schema/mvc http://www.springframework.org/schema/mvc/spring-mvc.xsd
        http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context http://www.springframework.org/schema/context/spring-context.xsd">

    <!-- DispatcherServlet Context: defines this servlet's request-processing
        infrastructure -->

    <!-- Enables the Spring MVC @Controller programming model -->
    <annotation-driven />
    <context:component-scan base-package="com.journaldev.spring" />

    <!-- Resolves views selected for rendering by @Controllers to .jsp resources
        in the /WEB-INF/views directory -->
    <beans:bean
        class="org.springframework.web.servlet.view.InternalResourceViewResolver">
        <beans:property name="prefix" value="/WEB-INF/views/" />
        <beans:property name="suffix" value=".jsp" />
    </beans:bean>
</beans:beans>
```

Le tre configurazioni trovate in questo file:

- **annotation-driven** abilita le annotazioni `@Controller`
- **context:component-scan** DispatcherServlet guarda per tutte quelle classi che sono annotate con `@Controller`.
- **InternalResourceViewResolver** trova le views presenti nel progetto, partendo da un percorso `"/WEB-INF/views"` e guardando l'estensione del file (.jsp).

Controller, Model e Views di questo esempio



Controller → Abbiamo un unico controller "HomeController.java" che gestisce le richieste sull'url "/" e "/user"

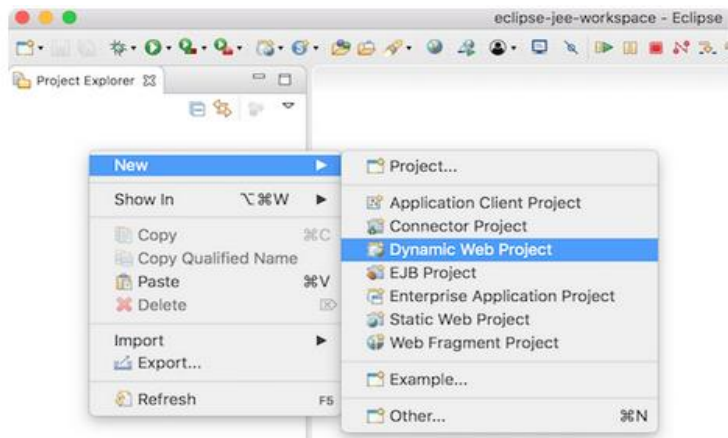
Views → home.jsp, user.jsp are the view pages in our spring mvc hello world example application.

Model → User.java sarà la model class che conterrà le informazioni dell'utente.

La struttura di cartelle del nostro progetto

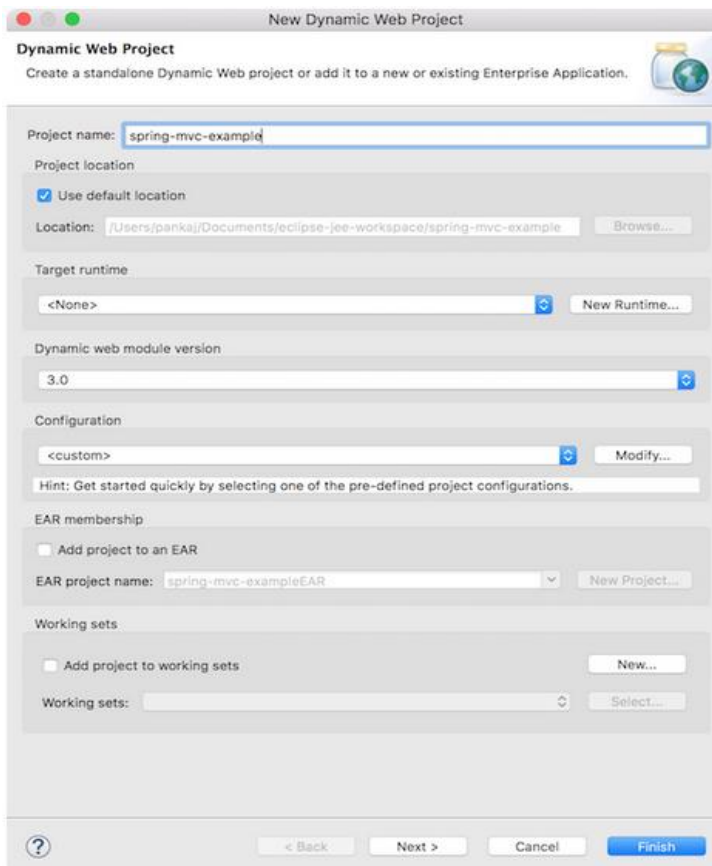
Se vogliamo creare un sito web dinamico, tradotto in inglese vogliamo un Dynamic Web Project; ma così facendo, noterete, che la struttura che vi troverete davanti sarà ben diversa da quella vista sopra. La prima cosa che salta all'occhio è la mancanza del file di configurazione pom.xml di maven.

Adesso passo-passo cerchiamo di ottenere lo stesso tipo di struttura.

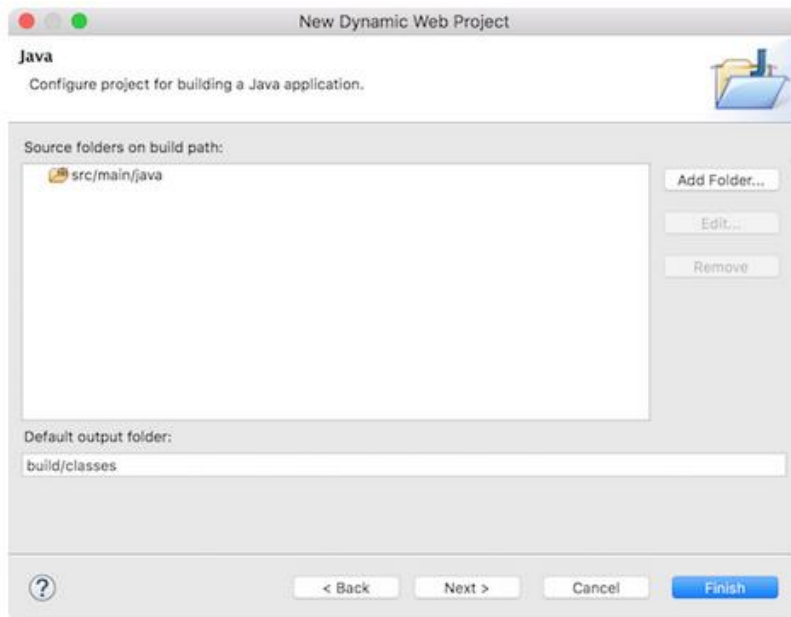


Configuration

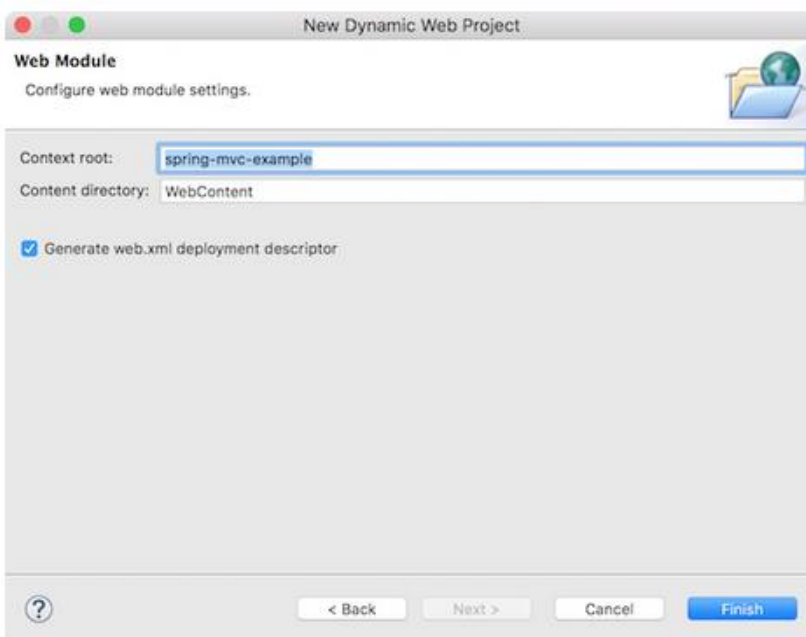
Oltre a dare un nome al progetto, possiamo voler dire su quale server deve essere eseguito il mio progetto (Target runtime), oppure aggiungere una serie di librerie. Possiamo sorvolare il discorso librerie perché le dipendenze le aggiungeremo successivamente con maven.



In questa pagina dovete rimuovere la cartella "src" e aggiungere la cartella "src/main/java".



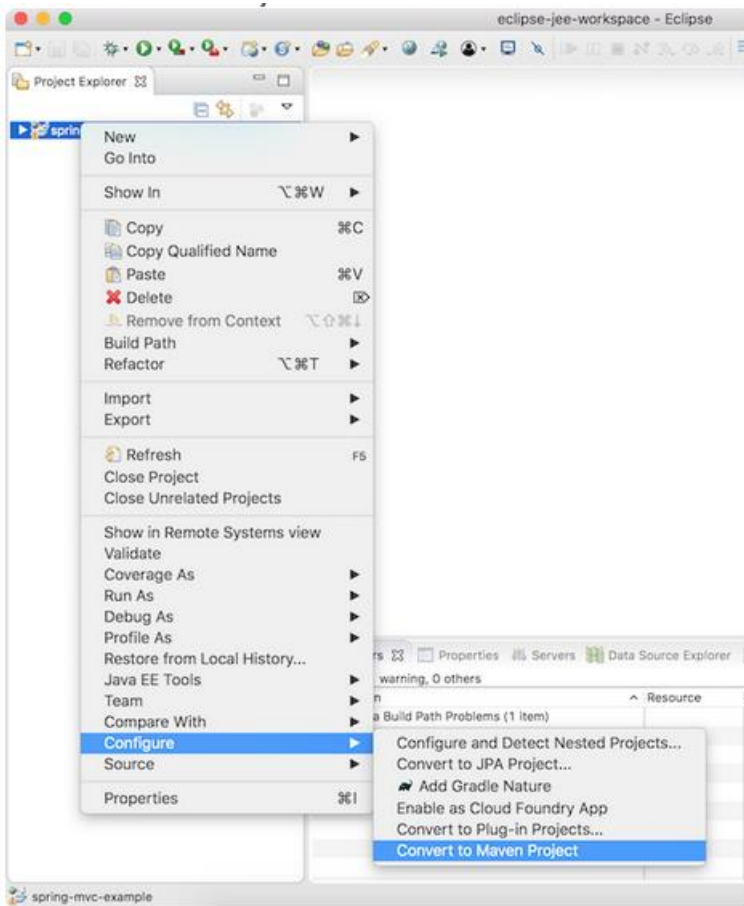
Successivamente, nell'immagine sotto, fornite il context root dell' applicazione, il nome, "spring-mvc-example" e assicuratevi di generare web.xml che sarà modificato come descritto [qui](#).



IL nostro progetto web è terminato.

Convertire il progetto Dynamic Web Project in Maven Project

In questo passaggio fondamentale otterremo la struttura vista all'inizio, dai un'occhiata [qui](#)



Coordinate del nostro progetto per creare il nostro pom.xml di configurazione del progetto

A screenshot of the 'Create new POM' wizard in Eclipse. The window title is 'Create new POM'. Below the title bar, it says 'Maven POM' and 'This wizard creates a new POM (pom.xml) descriptor for Maven.' The main area contains several input fields for project configuration: 'Project:' with the value '/spring-mvc-example', 'Artifact' section with 'Group Id:' 'com.journaldev.spring.mvc', 'Artifact Id:' 'spring-mvc-example', 'Version:' '0.0.1-SNAPSHOT', and 'Packaging:' 'war', 'Name:' 'Spring MVC Example', and 'Description:' 'Spring MVC Hello World Example'. At the bottom, there are three buttons: a help button with a question mark icon, a 'Cancel' button, and a 'Finish' button.

Spring MVC Dependencies to pom.xml

```
<!-- Add Spring Web and MVC dependencies -->
<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>4.3.9.RELEASE</version>
  </dependency>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-web</artifactId>
    <version>4.3.9.RELEASE</version>
  </dependency>
  <!-- Servlet -->
  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>servlet-api</artifactId>
    <version>2.5</version>
    <scope>provided</scope>
  </dependency>
  <dependency>
    <groupId>javax.servlet.jsp</groupId>
    <artifactId>jsp-api</artifactId>
    <version>2.1</version>
    <scope>provided</scope>
  </dependency>
  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>jstl</artifactId>
    <version>1.2</version>
  </dependency>
</dependencies>
```

Il nostro WAR file name prende il nome e la versione dalle coordinate del progetto
\${project.artifactId}

```
    </plugin>
  </plugins>
  <finalName>${project.artifactId}</finalName> <!-- added to remove Version from WAR file -->
</build>
</project>
```

Spring MVC Controller Class

```
package com.journaldev.spring.controller;

import java.text.DateFormat;
import java.util.Date;
import java.util.Locale;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

import com.journaldev.spring.model.User;

@Controller
public class HomeController {

    /**
     * Simply selects the home view to render by returning its name.
     */
    @RequestMapping(value = "/", method = RequestMethod.GET)
    public String home(Locale locale, Model model) {
        System.out.println("Home Page Requested, locale = " + locale);
        Date date = new Date();
        DateFormat dateFormat = DateFormat.getDateInstance(DateFormat.LONG, DateFormat.LONG, locale);

        String formattedDate = dateFormat.format(date);

        model.addAttribute("serverTime", formattedDate);

        return "home";
    }

    @RequestMapping(value = "/user", method = RequestMethod.POST)
    public String user(@Validated User user, Model model) {
        System.out.println("User Page Requested");
        model.addAttribute("userName", user.getUserName());
        return "user";
    }
}
```

Spring MVC Model Class

```
package com.journaldev.spring.model;

public class User {
    private String userName;

    public String getUserName() {
        return userName;
    }

    public void setUserName(String userName) {
        this.userName = userName;
    }
}
```

Spring MVC View Pages

home.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
<%@ page session="false"%>
<html>
<head>
<title>Home</title>
</head>
<body>
    <h1>Hello world!</h1>

    <p>The time on the server is ${serverTime}.</p>

    <form action="user" method="post">
        <input type="text" name="userName"><br> <input
            type="submit" value="Login">
    </form>
</body>
</html>
```

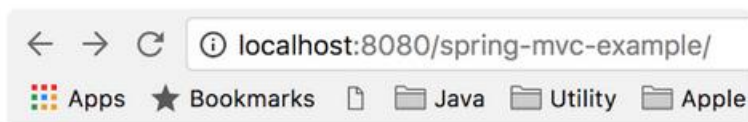
user.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>User Home Page</title>
</head>
<body>
<h3>Hi ${userName}</h3>
</body>
</html>
```

Esecuzione del progetto

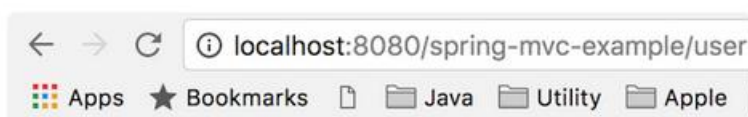
Quando mandi in esecuzione l'applicazione, run su Eclipse → java application;
automaticamente sarà deployata su tomcat :

<http://localhost:8080/spring-mvc-example/>



Hello world!

The time on the server is July 18, 2017 7:08:15 PM IST.



Hi Pankaj

Sommario

Introduzione.....	1
Spring MVC - Environment Setup.....	2
Step 1 - Setup Java Development Kit (JDK)	2
Step 2 – Install Apache Common Logging API	3
Step 3 - Setup Eclipse IDE	3
Step 4 - Setup Spring Framework Libraries	4
Spring IoC	4
La gestione dei Beans	4
SpringBoot	6
Primo progetto in Spring	7
Pom.xml e le dipendenze di Spring.....	7
La nostra demoApplication	7
SpringTransformer è il nostro Bean che deve essere automaticamente istanziato e per dire al framework di farlo va utilizzata l'annotazione @Component; invece per iniettare l'oggetto di tipo SpringTransformer nella classe CLI lo annoteremo con @Autowired.....	8
Springboot per la realizzazione di un servizio Rest	11
Controller @RestController	11
Lo status code	11
@PathVariable per aggiungere un parametro nella risposta.....	12
Altre informazioni recuperabili nella richiesta	12
Conversione automatica di un oggetto in JSON	13
Spring IOC	15
Cosa fa il proxy	17
PrototypeBean	18
proxyMode di tipo Interfaces	19
Bean scopes.....	20
Singleton vs Application	20
Insieme dei progetti a cui faccio riferimento nelle esercitazioni di Springboot.....	22
Esercitazioni di Spring Boot 1: anatomia di un progetto	22
File pom.xml	22
Classe Main che equivale alla Entrypoint del progetto.....	22
Conoscenza di alcune annotazione di Spring	22
Classe ServletInitializer.....	24
Application.properties	24
Esercitazioni di Spring Boot 2: il nostro primo controller REST	25

Modello dati	25
la classe PlayersController (@RestController).....	25
EndPoint “/players”	25
@RequestMapping, @GetMapping.....	25
Springboot : gestione della logica di business	26
controller REST	27
add	28
Update → richiesta in PUT	28
Delete → richiesta in delete.....	29
Spring Boot, come lavorare con i database	30
Intruduzione	30
Dipendenza al JPA	30
Application.properties parametri di connessione al database	30
SSL connection.....	31
Entità (POJO)	31
<i>Interazione con il database</i>	31
Inserimento di un calciatore da CURL	32
Creare un Dynamic Web Project con l'aggiunta di Spring	34
Architettura del progetto.....	34
dispatcherServlet nel file web.xml.....	34
Spring beans configuration	35
Controller, Model e Views di questo esempio	35
La struttura di cartelle del nostro progetto.....	36
Configuration.....	36
Convertire il progetto Dynamic Web Project in Maven Project	38
Coordinate del nostro progetto per creare il nostro pom.xml di configurazione del progetto	38
Spring MVC Dependencies to pom.xml.....	39
Spring MVC Controller Class	40
Spring MVC Model Class.....	40
Spring MVC View Pages.....	41
Esecuzione del progetto	41